

SOFTWARE TEST PLAN

Online Bookstore System

Buying & Selling Platform

Team: Team 4

Team Members:

1. PRUTHVIRAJ H – PES2UG24CS381
2. PUNEET LINGASHETTAR – PES2UG24CS382
3. PUNITH K – PES2UG24CS383
4. PURVITHA MACHIREDDY – PES2UG24CS384

1. INTRODUCTION

1.1 Purpose

This Test Plan describes how the Online Bookstore System will be tested to make sure that the developed system satisfies the requirements given in the Software Requirements Specification (SRS).

The system allows customers to search and purchase new or pre-owned books. It also allows users to submit books for selling. Administrators manage the catalog, inventory, orders, reviews and sales information. The system uses React for the frontend, Django REST Framework for the backend and PostgreSQL for storing data.

The main purpose of testing is to identify errors before the system is considered ready for submission and demonstration.(we must ensure that it is given equal priority)

1.2 Scope

Testing will cover the major functions of the Online Bookstore System, including:

- User registration and email activation
- User login and JWT authentication
- Password recovery
- Profile and address management
- Role-based access control
- Book searching and filtering
- Book details
- Shopping cart
- Wishlist
- Seller book listing
- Checkout
- Payment processing
- Order confirmation emails
- Order tracking
- Order history
- Admin catalog management
- Inventory management
- Review submission and moderation
- Sales dashboard
- Performance and concurrency
- Accessibility and responsive UI
- Database integrity
- Security requirements

These features are based on the functional, non-functional and security requirements defined in the SRS.

1.3 Objectives of Testing

The main objectives of testing are:

1. To verify that all functional requirements are working correctly.
2. To check that invalid inputs are handled properly.

3. To verify that different user roles have the correct permissions.
4. To check the integration between React, Django, PostgreSQL and external services.
5. To verify secure payment processing.
6. To check system performance under normal and concurrent usage.
7. To verify database consistency during order and inventory operations.
8. To check responsive behaviour on different screen sizes.
9. To identify and fix defects before final delivery.
10. To maintain traceability between requirements and test cases.

2. TEST ITEMS

The following parts of the system will be tested.

Test Item	Description
Authentication Module	Registration, activation, login, password reset and profile
Catalog Module	Search, filtering and book details
Cart Module	Add, update and remove cart items
Wishlist Module	Save and move books to cart
Seller Module	Book listing and validation
Order Module	Checkout, payment, email and order tracking
Admin Module	Catalog, inventory, reviews and analytics
Database	PostgreSQL data storage and transaction consistency
Security	Passwords, JWT, HTTPS, RBAC and payment tokenization
User Interface	Responsive layout and accessibility
Performance	Response time and concurrent users

The SRS specifies a React frontend communicating with a Django REST backend and PostgreSQL database, with external payment and email services.

3. FEATURES TO BE TESTED

This section lists the major features that will be tested.

3.1 User Authentication

The following will be tested:

- New user registration
- Email activation
- Login with valid credentials
- Login with invalid credentials
- JWT token generation
- Password reset
- Profile editing
- Address editing
- Customer, Seller and Admin permissions

For example, the SRS expects a newly registered account to remain inactive until email verification and expects invalid login credentials to return HTTP 401.

3.2 Book Search and Catalog

Testing will cover:

- Searching by book title
- Searching by author
- Searching by ISBN
- Searching by genre
- Price filtering
- Book condition filtering
- Rating filtering
- Sorting by price
- Sorting by popularity
- Opening a book details page
- Invalid book ID handling

The SRS specifies that search should return matching books within 2 seconds for a database containing 10,000 records.

3.3 Cart and Wishlist

Testing will verify:

- Adding a book to cart
- Updating quantity
- Removing a book
- Correct calculation of subtotal
- Correct calculation of taxes
- Wishlist persistence
- Moving wishlist items to cart
- Cart persistence for logged-in users

3.4 Seller Book Listing

The seller functionality will be tested for:

- Entering book information
- Selecting book condition
- Entering price
- Entering stock quantity
- Uploading cover images
- Required field validation
- Invalid image format handling
- Saving listings in Pending Approval state

The SRS defines the supported conditions as New, Like New, Good and Acceptable.

3.5 Checkout and Payment

Testing will include:

- Selecting shipping address
- Checking order summary
- Checking item quantities

- Checking taxes and shipping charges
- Calculating final amount
- Successful payment
- Failed payment
- Payment transaction ID
- Cart clearing after successful payment
- Payment gateway callback handling

The SRS specifies tokenized payment through a PCI-DSS-compliant gateway such as Stripe or Razorpay.

3.6 Order Management

Testing will cover:

- Order creation
- Order confirmation
- Email receipt
- Order tracking
- Order status changes
- Order history
- Invoice information

The expected order states are **Placed, Processing, Shipped, Delivered and Cancelled**.

3.7 Administration

Admin functions to be tested include:

- Adding books
- Editing books
- Deactivating books
- Deleting books
- Approving seller listings
- Updating stock
- Low-stock alerts

- Updating order status
- Review moderation
- Sales dashboard calculations

The SRS specifies that low-stock items should be flagged when stock is below 5 units.

4. FEATURES NOT TO BE TESTED

The following are outside the software boundary according to the SRS:

1. Physical book warehousing
2. Courier and physical delivery operations
3. Physical handling of books
4. Raw credit/debit card processing
5. Physical payment infrastructure

The system only handles the software side of payment processing through external payment gateways.

Third-party services such as payment gateways and email services will be tested only from the application's integration point.

5. TEST APPROACH

Testing will be carried out using a combination of manual and automated testing methods.

5.1 Security Validation

Security testing is an important part of this project because the application handles user accounts, personal information and payment-related data.

The following security requirements will be validated.

5.1.1 Password Security

Requirement: BOOK-SR-001

Testing will verify that:

- Passwords are not stored as plain text.
- Passwords are stored using secure hashing.
- Passwords are not printed in application logs.
- Database records contain password hashes instead of actual passwords.

The SRS specifies the use of Django's authentication mechanism with PBKDF2/bcrypt-style secure password hashing.

Test Case: TC-Sec-01

5.1.2 HTTPS and TLS

Requirement: BOOK-SR-002

Testing will verify:

- React-to-Django communication uses HTTPS.
- External service communication uses secure transport.
- TLS 1.2 or higher is enforced.
- Unencrypted HTTP requests are rejected or redirected as required.

Test Case: TC-Sec-02

The SRS specifically requires TLS 1.2/1.3 for REST API communication.

5.1.3 JWT Security

Requirement: BOOK-SR-003

Testing will verify:

- JWT tokens are signed.
- Access tokens expire after the specified period.

- Expired tokens cannot access protected APIs.
- Logged-out/revoked tokens cannot be reused.
- Unauthorized requests return HTTP 401.

The SRS specifies a 15-minute access-token expiry and server-side revocation after logout.

Test Case: TC-Sec-03

5.1.4 Role-Based Access Control

Requirement: BOOK-SR-004

Testing will verify that:

- Customers cannot access seller-only APIs.
- Customers cannot access admin APIs.
- Sellers cannot perform unauthorized admin operations.
- Admin operations are protected on the server side.
- Unauthorized requests return HTTP 403.

For example, a normal customer attempting to access `/api/admin/sales/` should be rejected.

Test Case: TC-Sec-04

5.1.5 Payment Data Security

Requirement: BOOK-SR-005

Testing will verify that:

- PAN is not stored in the local database.
- CVV is not stored in the local database.
- Card expiry information is not unnecessarily stored.
- Payment processing uses gateway tokens.
- Successful payments store the required transaction reference rather than raw card information.

The SRS explicitly requires payment tokenization through PCI-DSS-compliant gateways.

Test Case: TC-Sec-05

5.2 Functional Testing

Functional testing checks whether each feature performs according to its requirement.

The main method will be:

Input → System Processing → Expected Output

Both valid and invalid inputs will be used.

Example:

Test	Input	Expected Result
Login	Correct email/password	Login successful
Login	Wrong password	HTTP 401
Search	Existing book title	Matching books displayed
Search	Random text	No matching results
Cart	Add available book	Book added
Checkout	Valid address	Order summary displayed
Review	Delivered order	Review accepted
Review	Undelivered order	Review rejected

5.3 Integration Testing

Integration testing will verify communication between different parts of the system.

The following integrations will be tested:

- React ↔ Django REST API
- Django ↔ PostgreSQL
- Django ↔ Payment Gateway
- Django ↔ Email Service
- Payment Gateway ↔ Webhook
- Order Module ↔ Inventory Module

The SRS identifies Stripe/Razorpay, SMTP/SendGrid and PostgreSQL as important external interfaces.

5.4 Performance Testing

Performance testing will check whether the application responds within the specified time.

The following requirements will be checked:

- 95% of catalog/detail API requests should respond within 2 seconds.
- The application should support at least 500 concurrent active sessions.
- Response degradation should remain within the specified 20% limit.

JMeter can be used for load testing.

The SRS defines these performance targets under BOOK-NF-001 and BOOK-NF-003.

Test Cases:

- TC-Perf-01 – API response time
- TC-Perf-02 – 500 concurrent users

5.5 Reliability Testing

Reliability testing will check the availability of the application over time.

The target is:

Minimum monthly uptime = 99.5%

Scheduled maintenance is excluded from the calculation.

Automated health checks can be used to record availability.

Test Case: TC-Rel-01

5.6 Usability and Accessibility Testing

The interface will be tested on:

- Desktop – 1920×1080
- Tablet – 768×1024
- Mobile – 375×812

Testing will include:

- Page layout
- Navigation
- Button visibility
- Form usability
- Keyboard navigation
- Text readability
- Image alternative text
- Basic WCAG 2.1 AA requirements

Lighthouse can be used for the accessibility check. The SRS specifies a target score above 90/100 across the three device breakpoints.

Test Case: TC-UX-01

5.7 Database and Data Integrity Testing

PostgreSQL data will be checked to make sure that:

- User records are stored correctly.
- Books are stored correctly.
- Cart information is maintained.
- Orders are linked to the correct users.
- Inventory values are updated correctly.
- Reviews belong to the correct users/books.
- Payment audit information is stored correctly.

The system will also test concurrent purchases of the last available book.

Expected result:

One purchase succeeds and the other receives an out-of-stock response.

This validates the ACID transaction requirement in the SRS.

Test Case: TC-Int-01

5.8 Regression Testing

Regression testing will be performed after important changes or bug fixes.

For example, after changing the checkout module, the team will again check:

- Cart
- Checkout
- Payment
- Order creation
- Inventory update
- Email confirmation

This is done to make sure that fixing one part does not break another part.

6. TEST ENVIRONMENT

Frontend

- React.js
- Modern web browser

Backend

- Django REST Framework
- Python 3.11+

Database

- PostgreSQL 15+

Deployment/Infrastructure

- Linux-based environment
- Docker
- AWS EC2 or Render

Testing Tools

The following tools may be used:(As of today and might be changed as per the requirement)

Tool	Purpose
Postman	REST API testing
JMeter	Performance/load testing
Browser DevTools	Frontend and network testing
Lighthouse	Accessibility testing
PostgreSQL tools	Database verification
Git	Version control
Docker	Testing the application environment

8. PASS / FAIL CRITERIA

8.1 Pass Criteria

A test case will be considered **Pass** when:

- The actual result matches the expected result.
- No critical error occurs.
- The requirement is satisfied.
- The required response code is received.
- Required database changes are correct.

8.2 Fail Criteria

A test case will be considered **Fail** when:

- The actual result does not match the expected result.

- The application crashes.
- Incorrect data is stored.
- Unauthorized access is allowed.
- A security requirement is violated.
- Performance exceeds the specified limit.

9. DEFECT MANAGEMENT

When a defect is found, it will be recorded with the following information:

- Defect ID
- Test Case ID
- Module
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Assigned team member
- Status

Example

Field	Example
Defect ID	BUG-001
Module	Login
Test Case	TC-Auth-02
Description	Invalid password allows login
Expected	HTTP 401
Actual	Login successful
Severity	High
Status	Open

After fixing a defect, the related test case will be executed again.

10. TEST DELIVERABLES (MIGHT GET CHANGED AS PER REQUIREMENT)

The following documents and results will be maintained during testing:

1. Test Plan
2. Test Cases
3. Test Execution Results
4. Defect/Bug Report
5. Performance Test Results
6. Security Validation Results
7. Accessibility Test Results
8. Database Validation Results
9. Requirements Traceability Matrix
10. Final Test Summary

11. REQUIREMENTS TRACEABILITY

The test plan will maintain traceability from:

Requirement → Design Module → Test Case → Test Result

The SRS already provides an RTM connecting all requirements with their corresponding test cases.

11.1 Functional Requirement Traceability

Requirement Group	Test Cases
BOOK-F-001 to BOOK-F-005	TC-Auth-01 to TC-Auth-05
BOOK-F-006 to BOOK-F-010	TC-Cat-01 to TC-Cat-05
BOOK-F-011	TC-Sel-01
BOOK-F-012 to BOOK-F-016	TC-Ord-01 to TC-Ord-05
BOOK-F-017 to BOOK-F-020	TC-Adm-01 to TC-Adm-04

This provides coverage for all **20 functional requirements**.

11.2 Non-Functional Requirement Traceability

Requirement	Test Case
BOOK-NF-001 – Response Time	TC-Perf-01
BOOK-NF-002 – Uptime	TC-Rel-01
BOOK-NF-003 – Concurrent Users	TC-Perf-02
BOOK-NF-004 – UI/Accessibility	TC-UX-01
BOOK-NF-005 – Data Retention	TC-Dat-01
BOOK-NF-006 – ACID/Data Integrity	TC-Int-01

The SRS defines six NFRs covering performance, reliability, scalability, accessibility, data retention and transaction integrity.

11.3 Security Requirement Traceability

Requirement	Test Case	Main Validation
BOOK-SR-001	TC-Sec-01	Password hashing
BOOK-SR-002	TC-Sec-02	HTTPS/TLS
BOOK-SR-003	TC-Sec-03	JWT expiry/revocation
BOOK-SR-004	TC-Sec-04	Server-side RBAC
BOOK-SR-005	TC-Sec-05	Payment tokenization

These five security requirements are explicitly mapped to security test cases in the SRS.

12. TEST EXIT CRITERIA

Testing will be considered complete when the following conditions are satisfied:

1. All 20 functional requirements have been tested.
2. All 6 non-functional requirements have been validated.
3. All 5 security requirements have been checked.
4. Major defects have been fixed or properly documented.
5. No critical security vulnerability remains.
6. Performance requirements have been checked.

7. The RTM has been updated with test results.
8. Required acceptance test suites have been completed.

The SRS specifies 100% coverage of the 20 functional requirements, validation of all six NFRs, successful security checks for all five security requirements, and an updated RTM as project exit criteria.

13. ACCEPTANCE TEST SUITES

The project will use the following five acceptance test suites.

Suite 1 – User Authentication and Access

Tests:

- Registration
- Email activation
- Login
- Password reset
- RBAC

Suite 2 – Catalog and Search

Tests:

- Book search
- Filters
- Sorting
- Book details
- Seller book submission

Suite 3 – Checkout and Payment

Tests:

- Address selection
- Cart synchronization

- Payment gateway
- Tokenized payment
- Order confirmation email

Suite 4 – Admin and Fulfillment

Tests:

- Catalog approval
- Inventory
- Order status
- Review moderation
- Sales dashboard

Suite 5 – Quality and Performance

Tests:

- JMeter load test
- 500 concurrent users
- WCAG 2.1 AA checks
- Lighthouse accessibility audit
- SSL/TLS security validation

These five acceptance suites are directly defined in the SRS.

14. CONCLUSION

This Test Plan provides a simple and structured approach for testing the Online Bookstore System.

The testing process covers the main user, seller and administrator functions along with performance, reliability, accessibility, database integrity and security.

The Requirements Traceability Matrix will be used to make sure that every requirement has a corresponding test case. Testing will continue until the defined functional, non-functional and security requirements have been verified and the project exit criteria have been satisfied.

The final objective is to ensure that the Online Bookstore System is working correctly, securely and consistently according to the requirements specified in the SRS.

THANK YOU

-TEAM 4